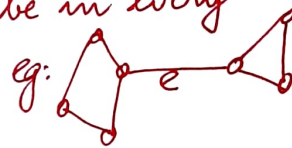
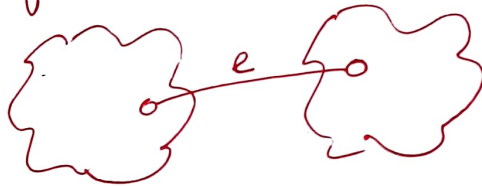


Roll No:

Name:

Q1. Suppose  $G$  is a weighted, connected, undirected, simple graph and  $e$  is a largest-weight edge in  $G$ . Can there be a minimum spanning tree of  $G$  that contains  $e$ ? If your answer is "Yes" sketch such a graph. If your answer is "No", prove it. [4 marks]

A bridge is an edge the removal of which disconnects the graph. If a largest-weight edge of  $G$  happens to be a bridge of  $G$  then it will have to be in every MST of  $G$



Answer: Yes

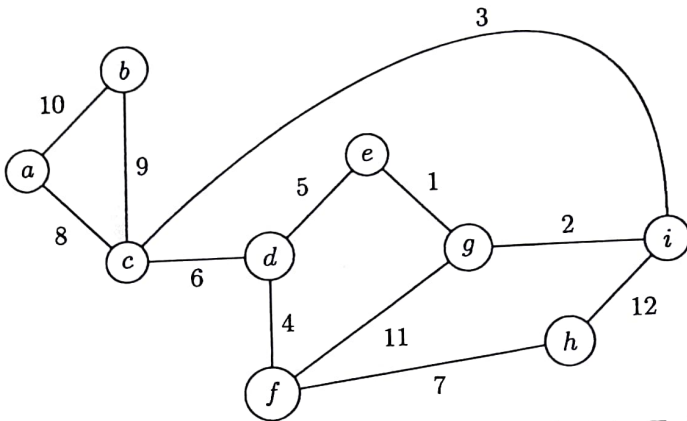
Q2. Sort the following  $4 \times 4$  mesh using the parallel Snake-like sorting algorithm discussed in the class. Show the steps. [4 marks]

|    |    |    |    |
|----|----|----|----|
| 08 | 06 | 11 | 12 |
| 07 | 13 | 02 | 05 |
| 14 | 01 | 15 | 04 |
| 09 | 16 | 10 | 03 |

|      |    |    |    |
|------|----|----|----|
| 6    | 8  | 11 | 12 |
| 13   | 7  | 5  | 2  |
| 1    | 4  | 14 | 15 |
| 16   | 10 | 9  | 3  |
| ↔↔↔↔ |    |    |    |
| 1    | 4  | 5  | 2  |
| 6    | 7  | 9  | 8  |
| 13   | 8  | 11 | 12 |
| 16   | 10 | 14 | 15 |
| ↓↓↓↓ |    |    |    |
| 1    | 2  | 4  | 5  |
| 9    | 7  | 6  | 3  |
| 8    | 11 | 12 | 13 |
| 16   | 15 | 14 | 10 |
| ↔↔↔↔ |    |    |    |
| 1    | 2  | 4  | 3  |
| 8    | 7  | 6  | 5  |
| 9    | 11 | 12 | 10 |
| 16   | 15 | 14 | 13 |
| ↓↓↓↓ |    |    |    |

|    |    |    |    |   |
|----|----|----|----|---|
| 1  | 2  | 3  | 4  | → |
| 8  | 7  | 6  | 5  | ← |
| 9  | 10 | 11 | 12 | → |
| 16 | 15 | 14 | 13 | ← |

Q3. Consider the following graph.



(a) Show the sequence in which edges are added to  $T$ , if  $T$  is the MST calculated by Kruskal's algorithm [3 marks]

~~-|a|b|c|d|e|f|g|h|i~~  
~~0000000000~~  
~~-|a|b|c|d|e|f|g|h|i~~  
~~1080000000~~  
~~-|a|b|c|d|e|f|g|h|i~~  
~~96000003~~  
~~ac|b|d|e|f|g|h|i~~  
~~ac|9600212~~  
~~ac|g|b|d|e|f|h~~  
~~ac|9611112~~  
~~ac|g|951112~~  
~~ac|g|e|b|f|h~~  
~~ac|g|e|9412~~  
~~ac|g|e|d|b|h~~  
~~ac|g|e|d|97~~  
~~ac|g|e|d|f|h~~  
~~ac|g|e|d|f|9~~  
 The edge sequence:  $ac, ci, ig, ge, ed, df, fh, cb$

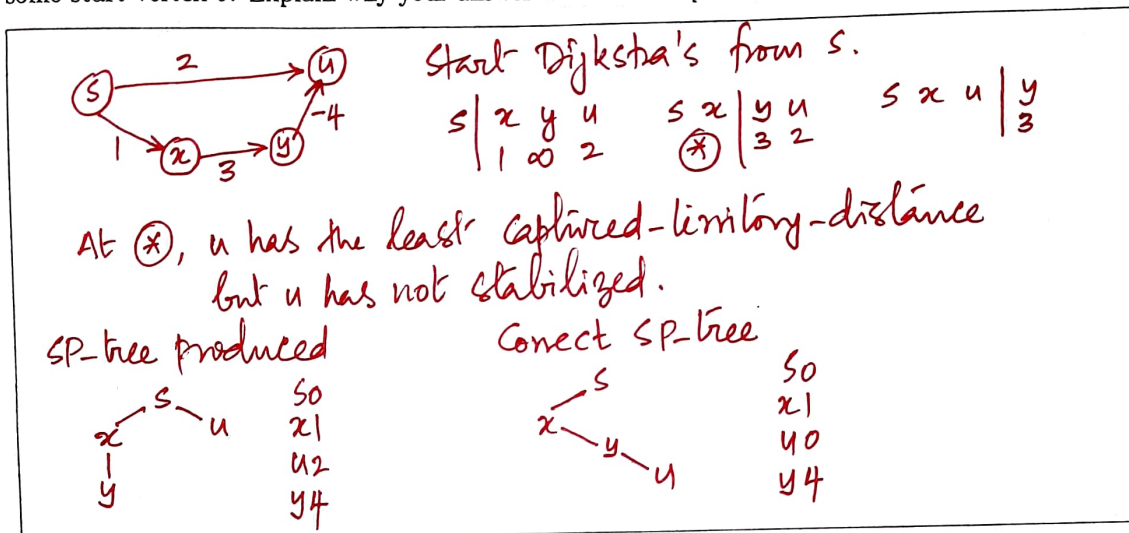
(b) Show the sequence in which edges are added to  $T$ , if  $T$  is the MST calculated by Prim's algorithm with vertex  $a$  as the source. [3 marks]

edge sequence: 1, 2, 3, 4, 5, 7, 8, 9  
 6 forms a cycle with 5, 1, 2, 3. 10 forms a cycle with 8, 9.  
 11 forms a cycle with 1, 5, 4. 12 forms a cycle with 7, 4, 5, 1, 2

Q4. If Dijkstra's algorithm is run on the graph of Question 3 with vertex  $g$  as the source, what is the sequence in which the vertices are captured? [4 marks]

~~g|a|b|c|d|e|f|h|i~~  
~~112~~  
~~g|e|a|b|c|d|f|h|i~~  
~~6112~~  
~~g|i|a|b|c|d|f|h~~  
~~56114~~  
~~g|e|i|a|b|c|d|f|h~~  
~~0125131461114~~  
~~g|e|i|c|d|a|b|f|h~~  
~~0125613141014~~  
~~g|e|i|c|d|f|a|b|h~~  
~~01256131414~~  
 The sequence of capture:  $g, e, i, c, d, f, a(b, h/h, b)$

Q5. Give an example of a weighted directed graph,  $G$ , with negative-weight edges, but no negative-weight cycle, such that Dijkstra's algorithm incorrectly computes the shortest-path distances from some start vertex  $v$ . Explain why your answer is correct. [8 marks]



Q6. Suppose that every shortest path from some vertex  $v$  in an  $n$ -vertex weighted graph  $G$  to any other vertex in  $G$  has at most  $k < n - 1$  edges. Show that it is sufficient to run the Bellman-Ford algorithm for only  $k$  iterations, instead of  $n - 1$ , to solve the single-source shortest-paths problem for  $v$  in  $G$ . [4 marks]

for  $i = 1$  to  $k$   
 for  $(u, v) \in E$   
 Relax( $u, v, w$ )

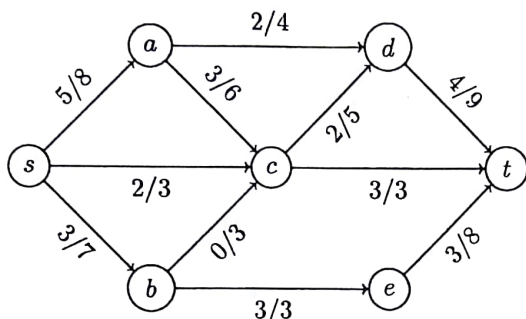
Let  $(s, u_1) - (u_1, u_2) - \dots - (u_{r-1}, u_r)$  be a SP where  $r \leq k$ .  $(s, u_1)$  is relaxed in the 1st iteration. So  $u_1$  stabilizes after iteration 1.  $u_i$  stabilizes after iteration  $i$ .  $u_1, \dots, u_k$  all stabilize in the first  $r$  iterations. They don't change afterwards

Q7. Define  $A_0(j) = j + 1$  and for  $k \geq 1$ ,  $A_k(j) = A_{k-1}^{(j+1)}(1)$ . Let  $\text{LEVEL}(x)$  be the largest  $k$  such that  $x.p.r \not\geq A_k(x.r)$ . Let  $\text{ITER}(x)$  be the largest  $i$  such that  $x.p.r \not\geq A_{\text{LEVEL}(x)}^{(i)}(x.r)$ . If  $x.r = 10$  and  $x.p.r = 1400$ , then compute  $\text{LEVEL}(x)$  and  $\text{ITER}(x)$ . [4 marks]

$A_0(j) = j + 1$ .  $A_1(j) = j + 2$ .  $A_2(j) = 2j + 3$   
 $A_2^{(i)}(j) = 2^i j + 3 \cdot 2^{i-1} + \dots + 3 = 2^i j + 3(2^i - 1) = 2^i(j + 3) - 3$   
 $A_3(j) = 2^{j+3} - 3$   
 $A_0(10) = 11, A_1(10) = 12, A_2(10) = 23, A_3(10) = 8189$ .  $\text{LEVEL}(x) = 2$   
 Apply  $A_2$  repeatedly on 10: 23, 49, 101, 205, 413, 829, 1661  
 $\text{ITER}(x) = 6$



Q8. Answer the following questions on the flow network  $N$  and flow  $f$  shown below: (a) What are the forward and backward edges of the augmenting path  $sbcadt$ ? (b) How many augmenting paths are there with respect to flow  $f$ ? For each such path, list the sequence of vertices of the path and the residual capacity of the path. (c) What is the value of a maximum flow in  $N$ ? [6 marks]



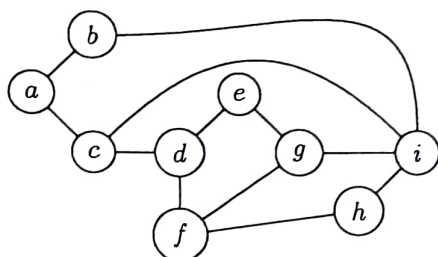
- a) In  $sbcadt$ ,  $ca$  is backward, the rest are forward
- b) As  $be$  and  $ct$  are saturated, all augmenting paths are through  $d$ . Using all permutations of  $\{a, b, c\}$  we have  ~~$sdt$~~ ,  $scdt(1)$ ,  $sbdct$ ,  $sadct(2)$ ,  ~~$sabct$~~ ,  ~~$sbadct$~~ ,  $sacdt(3)$ ,  $scadt(1)$ ,  ~~$sbedct(3)$~~ ,  ~~$sabedct$~~ ,  ~~$sacbedct$~~ ,  $sbcadt(2)$ ,  ~~$sbaedct$~~ ,  ~~$scaebct$~~ ,  ~~$scaedct$~~ ,  ~~$sbcadct(3)$~~ . There are 6 augmenting paths.
- c) Max flow = 15. (Use  $sbcadt$  and then  $sacdt$  as aug. paths)

Q9. Solve the following recurrence relation:  $T(1) = 1$ . For  $n > 1$ ,  $T(n) = 2T(n/2) + n \log(\log n)$ . [4 marks]

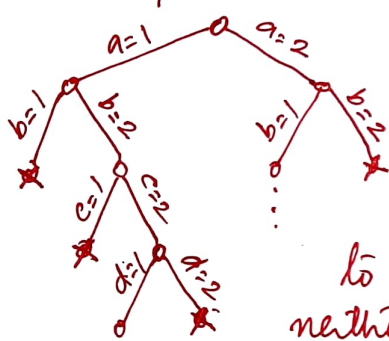
Master's Thm does not apply, as  $n \log \log n$  is  $\omega(n)$  and  $o(n^{1+\epsilon})$  for constant  $\epsilon$

$$\begin{aligned}
 T(n) &= 2T\left(\frac{n}{2}\right) + n \log \log n = 4T\left(\frac{n}{4}\right) + n \log \log \frac{n}{2} + n \log \log n \\
 &= 2^k T\left(\frac{n}{2^k}\right) + n \left[ \log \log \frac{n}{2^{k-1}} + \dots + \log \log \frac{n}{2} + \log \log n \right] \\
 &= n T(n) + n \cdot \log \left[ (\log n) \cdot (\log n - 1) \cdot \dots \cdot 2 \cdot 1 \right] \quad /* k = \log n */ \\
 &= n + n \log \left[ (\log n)! \right] \\
 &= \Theta(n \cdot \log n \cdot \log \log n) \quad /* \text{using Stirling's approx} */
 \end{aligned}$$

Q10. Run the backtracking algorithm on the following graph to vertex colour it with two colours (1 and 2), starting at vertex  $a$ . Each vertex has its neighbours listed in alphabetical order. Explain the results. [4 marks]



state space



The backtracking algorithm attempts a DFS of this state space tree, except that it prunes subtrees found to be invalid. (X). The algorithm proceeds

to  $a_1 b_2 c_2 d_1 e_2 f_2 g_1 h_1$ . Now neither 1 nor two works for  $i$ . The right side is a mirror image of the left side. So the algorithm fails to find a valid colouring.

Q11. For all  $n \geq 1$ ,  $T(n, n) = 1$ .

For all  $r > 1$  and  $n > 1$ ,  $T(n, r) = T(n/r, r^2) + 1$ . Compute  $T(n, 2)$ . [4 marks]

Assume :  $2 \leq r \leq n$

$$\begin{aligned} T(n, r) &= T\left(\frac{n}{r}, r^2\right) + 1 = T\left(\frac{n}{r^3}, r^8\right) + 3 \\ &= T\left(\frac{n}{r^{2^k-1}}, r^{2^k}\right) + k = k+1 \text{ when } \frac{n}{r^{2^k-1}} = r^{2^k}. \end{aligned}$$

$$\text{That is, } n = r^{2^{k+1}-1} \Rightarrow 2^{k+1}-1 = \log_r n$$

$$\Rightarrow k = O(\log_2 \log_r n)$$

$$\text{So, when } r=2, \quad T(n, r) = O(\log_2 \log_2 n)$$

Q12. Professor Can't-Dress-Like-Superman must put on underpants, socks, wrist-watch, pants, shoes, shirt, belt, tie, and coat. Construct a DAG  $G$  for the professor: each directed edge  $(u, v)$  means that garment  $u$  MUST be put on before garment  $v$ . Topologically sort  $G$  using the algorithm discussed in the class. Show the steps. [4 marks]

Q13. An  $O(n^3)$  time randomized algorithm for a minimization problem  $P$  finds the least cost solution with a probability of at least  $1/n^2$ . Show that there exists an  $O(n^6)$  time randomized algorithm that finds the least cost solution with a probability of at least  $1 - 1/e^n$ . [4 marks]

Pr. [one execution fails to produce the least cost soln]  $\leq 1 - \frac{1}{n^2}$

Repeat  $n^2$  times.

Pr [all  $n^2$  executions fail]  $\leq \left(1 - \frac{1}{n^2}\right)^{n^2} < \frac{1}{e}$ .

Repeat the whole  $n$  times

Pr [all repetitions fail]  $\leq \frac{1}{e^n}$ .

Time taken =  $O(n^3) \times n^2 \times n = O(n^6)$